

Homework 01

Due: 11:59pm, Thursday, May 28, 2026

Focus:

- vectors
- strings
- functions
- Standard I/O
- Clean code!

Submission instructions:

- You should submit your homework on [Gradescope](#).
- For this assignment you should turn in one single source file named: `hw1.cpp`
- The C++ file should contain a header comment block as follows:

```
/*
Author: [Your name here]
Assignment: HW1
Date due: MM DD
I pledge that I have completed this assignment without the use of AI tools or
collaborating with anyone else, in conformance with the NYU School of
Engineering Policies and Procedures on Academic Misconduct.
*/
```

Note:

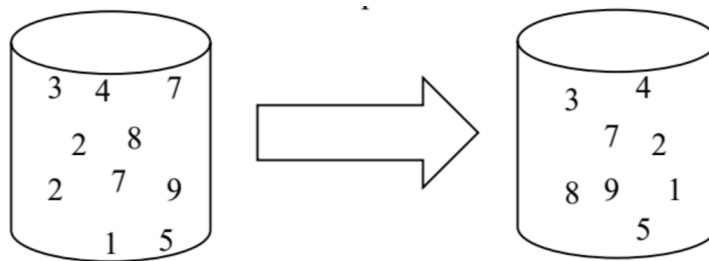
- **Late submissions** will be accepted within a 24-hour window with a **25% penalty**. Submissions beyond this timeframe will receive a grade of **zero**.
- **Plagiarism** or academic dishonesty will, at least, result in a zero for the assignment for the first offense, and failure of the course for subsequent offenses.
- *It is important that the work required by the course is yours. You should not use **ChatGPT** or other **AI tools** for any purpose other than idea generation.*

REMINDER: do not use additional library features that we have not covered. For this specific assignment, you may use everything we have learned up to the release date of this assignment. Please reach out to us if you're at all unsure about any instruction or whether a C++ structure is or is not allowed.

Question 01:

- 1) Design a C++ function named `isInVector` that efficiently determines whether a given integer exists within a provided vector or array of integers. The function should accept two parameters: the vector/array of integers and the integer to search for. It should return a boolean value indicating whether the searched integer is present in the provided data structure.
- 2) Develop a C++ function named `makeUnique` that efficiently creates a new vector containing unique elements from a given vector or array of integers. The function should accept two parameters: the original vector/array and an empty vector where the unique elements will be stored. Upon execution, `makeUnique` should populate the second vector with the integers from the first one, removing any duplicate entries.

An example illustration demonstrates the input vector on the left and the resultant unique-filled vector on the right.



- 3) Implement a C++ function called `fillVectFromFile` designed to populate a vector or array of integers with data retrieved from a user-selected input file. The function prompts the user for a file name and ensures that the file opens correctly before proceeding. If the file fails to open, the function terminates gracefully without any further execution.
- 4) Using the provided functions, solve the following problem: Obtain a set of integers stored in a file specified by the user. Write these integers into a file named `unique_ints`, ensuring that duplicate entries are eliminated.

Your main function should solely create variables and call functions to address the problem. There must be NO INPUT and NO OUTPUT within the main function.

Good Code

Writing good code means that your program needs:

- Good comments.
 - Every program file should have a **comment at the top** to identify the file, the author and, *briefly*, the purpose of the code in the file.
 - Every function definition should have a comment identifying its purpose, the purpose of its parameters and that of its return value. Unlike Python docstrings, this comment comes just *before* the function.
 - Any block of code, e.g. loop, should have a comment to explain enough of what it does so that the reader could skip reading the code in that block altogether without missing anything.
- Good functions. Well, we addressed that above in the requirements. Normally, thinking of good functions is a key part of your task. You are welcome to add any functions you think would be helpful here!
- **Function prototypes**
 - Function prototypes come *before* main.
 - Function definitions should come *after* main.
 - If you haven't covered function prototypes, don't worry, it will only take you a few minutes to add them to your program.
- **Good naming.** Both variables and functions should have good names.
 - A function's name should say clearly *what it does*.
 - A variable's name should say clearly what it holds. No, **not its type** but what its *purpose* is in the application.
 - Calling a vector of strings something like "stringVec" would be silly at best. If the vector is holding the lines from a file, perhaps it might be called "lines".
 - In general, single letter names are unacceptable.
 - Yes, we do often use the letters **i** and **j** as loop index variables, at least if the loop is reasonably short and their scope is limited by the loop itself. Even then, a more meaningful name helps clarify at a glance the intent of the code. An index variable called **row** might work better in some cases.
 - Similarly, variables with names like **temp** don't help the reader and should (almost) never be used.
- Good format.
 - It should be easy to see the *structure* of your program even if it was held far enough away that you could not read the code itself. Unlike Python, C++ does not *require* indentation. But good indentation makes code easier to read. And writing code that is easy to read is one of the most important principles of good programming
 - **Line length.** Long lines have two problems. First, unless you are reading the code on a device with a wide enough window, there is going to be annoying line wrap (aargh!). And secondly, it just makes it harder to see the logic that must be

embedded in there. A good rule of thumb is to assume your program will eventually get printed and due to whatever limitations, anything over 80 characters in a line will result in line wrap.

- **Function length.** Years ago, the recommended length limit was about 24 lines. That was how much we could fit on our terminal screens! :-). It is still a common recommendation to keep functions short for readability, but we will be a little more tolerant and allow functions to be as long as 40 lines.
- **Whitespace.** It usually makes code more readable if you put whitespace around your operators. Especially when there is a lot going on on the line. A simple example would be that `cin >> word`, is definitely more readable than `cin>>word`
- Some people will insist that where you place the opening curly brace for a block is extremely important. { Wars have been waged over such! } I have my preferences, but I won't force you to share them. Just be consistent.
- Global variables. **Don't.** No program that you will write for this course will need global variables. Do not use them.
 - Global *constants*, on the other hand, are highly recommended. (Remember that GLOBAL_CONSTANTS are written with all caps.)

Please, refer back to these guidelines throughout the semester. If you violate any of them, you may be penalized. And we much rather give out perfect scores!